Java Web Development Assignment — Level 2

Project: Smart Expense Manager Pro

Difficulty: Medium to Advanced
Estimated Duration: 5–7 hours

Technology Stack: Java, Jakarta Servlet, JDBC, MySQL, HTML, CSS, Apache Tomcat

1. Project Objective

Upgrade the Level 1 Expense Tracker into a more powerful expense management application.

The application should allow users to manage expenses, set monthly budgets, analyze spending patterns, and identify where their money is going.

2. Main Menu

Keep the home page short and simple:

1. Add Expense
2. Manage Expenses
3. Search & Filter
4. Financial Dashboard

3. Functional Requirements

Feature 1: Add Expense

Create an HTML form containing:

- Expense Title
- Amount
- Category: Food, Travel, Education, Shopping, Bills, Other
- Expense Date
- Payment Method: Cash, UPI, Card
- Description (optional)

Requirements:

- Validate all mandatory fields on the server.
- Amount must be greater than zero.
- The date must not be invalid.
- Save the record in MySQL using JDBC.
- Display a success or error message.

Feature 2: Manage Expenses

Display all expenses in a dynamically generated HTML table.

Columns:

`ID | Title | Amount | Category | Date | Payment Method | Actions`

Each record must support:

- **Edit:** Load the selected expense into an HTML form with existing values.
- **Update:** Save the modified values to MySQL.
- **Delete:** Remove the selected expense after confirmation.

Requirements:

- Use the expense ID to identify records.
- Handle invalid or non-existing IDs.
- Use POST requests for update and delete operations.
- Redirect after successful modification to prevent duplicate submissions.

Feature 3: Search & Filter

Allow users to search and filter expenses using multiple criteria:

- Expense title or keyword
- Category
- Payment method
- Start date
- End date
- Minimum amount
- Maximum amount

Requirements:

- Users must be able to apply multiple filters simultaneously.
- All filters must be optional.
- Validate date ranges and amount ranges.
- Use `PreparedStatement` to prevent SQL injection.
- Display matching results in a table.
- Display `No Expenses Found` when no records match.

Example: Find all UPI payments in the Food category between ₹100 and ₹1,000 made during a particular month.

Feature 4: Financial Dashboard

Create a dashboard using actual database records.

Display these metrics:

1. Total expenses for the selected month.
2. Number of transactions.
3. Highest individual expense.
4. Average transaction amount.

5. Category with the highest total spending.
6. Remaining monthly budget.

Also display a category-wise spending report.

Example:

```
FINANCIAL DASHBOARD — OCTOBER

Total Spent          : ₹12,500
Transactions         : 24
Highest Expense      : ₹3,000
Average Transaction  : ₹520.83
Top Category         : Shopping
Monthly Budget       : ₹15,000
Remaining Budget     : ₹2,500
```

The values above are examples only. Calculate all values dynamically from the database.

Feature 5: Monthly Budget Management

Allow the user to set a budget for a particular month.

Requirements:

- Enter the month and budget amount.
- Save the budget in MySQL.
- Allow the budget to be updated.
- Calculate actual spending for that month.
- Display the remaining budget.
- Show a warning when spending reaches 80% of the budget.
- Show an exceeded-budget warning when spending goes above 100%.

Example:

```
Monthly Budget: ₹10,000
Current Spending: ₹8,500

Warning: You have used 85% of your monthly budget.
Remaining Budget: ₹1,500
```

4. Database Design

Create a database named `expense_manager_pro`.

```
CREATE DATABASE expense_manager_pro;
USE expense_manager_pro;
```

```sql
CREATE TABLE expenses (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    category VARCHAR(50) NOT NULL,
    expense_date DATE NOT NULL,
    payment_method VARCHAR(20) NOT NULL,
    description VARCHAR(255)
);

CREATE TABLE monthly_budgets (
    id INT PRIMARY KEY AUTO_INCREMENT,
    budget_month CHAR(7) NOT NULL UNIQUE,
    budget_amount DECIMAL(10,2) NOT NULL
);
```

The `budget_month` column should use the `YYYY-MM` format, such as `2026-10`.

5. Technical Requirements

1. Use `HttpServlet` for request handling.
2. Use `doGet()` for retrieving and displaying information.
3. Use `doPost()` for operations that modify database records.
4. Use JDBC directly inside the Servlet classes.
5. Use `DriverManager`, `Connection`, `PreparedStatement`, and `ResultSet`.
6. Use `executeUpdate()` for INSERT, UPDATE, and DELETE operations.
7. Use `executeQuery()` for SELECT and aggregate queries.
8. Use SQL aggregate functions such as `SUM()`, `COUNT()`, `MAX()`, and `AVG()`.
9. Use `GROUP BY` to calculate category-wise spending.
10. Use try-with-resources to close JDBC resources.
11. Validate all user input on the server.
12. Handle SQL exceptions and invalid input gracefully.
13. Generate dynamic HTML using `response.getWriter()`.
14. Use `response.sendRedirect()` after successful form submissions.
15. Escape user-provided text before displaying it in generated HTML.

Restrictions: No JSP, DAO, Spring, Hibernate, or ORM. All database connections and SQL operations must be written directly in the relevant Servlet classes.

6. Suggested Project Structure

```
ExpenseManagerPro/
│
├── src/main/java/
│   └── com/expense/
│       ├── AddExpenseServlet.java
│       ├── ManageExpenseServlet.java
│       ├── UpdateExpenseServlet.java
```

```
│       ├── DeleteExpenseServlet.java
│       ├── SearchExpenseServlet.java
│       ├── DashboardServlet.java
│       └── BudgetServlet.java
│
└── src/main/webapp/
    ├── index.html
    ├── addExpense.html
    ├── searchExpense.html
    └── css/
        └── style.css
```

Students may create additional HTML files as needed. Editing forms, expense tables, and dashboard results must be generated by Servlets rather than JSP.

7. Mandatory Test Cases

Test Scenario	Expected Result
Add a valid expense	Record saved
Add a negative amount	Validation error
Edit an existing expense	Record updated
Edit an invalid ID	Appropriate error
Delete an expense	Record removed
Apply multiple search filters	Correct matching records
Enter start date after end date	Validation error
Set a monthly budget	Budget saved
Spend 85% of the budget	Warning displayed
Exceed the budget	Budget exceeded warning
Search with no matching results	Appropriate message

8. Bonus Challenges

Complete any one challenge after the mandatory features:

- **Spending Trends:** Compare this month's expenses with the previous month.
- **Top 3 Expenses:** Display the three most expensive transactions.
- **CSV Export:** Generate a downloadable CSV report of filtered expenses.
- **Recurring Expense Detector:** Identify repeated expense titles and summarize their total cost.

9. Submission Requirements

Submit:

1. Complete source code.
2. SQL database creation script.
3. Screenshots of all four menu options.
4. Screenshots showing edit, delete, filtering, and budget alerts.
5. A short explanation of the SQL queries used for the financial dashboard.

10. Evaluation Criteria — 100 Marks

Category	Marks
JDBC connectivity and database operations	10
Add, edit, update, and delete	20
Search and multi-condition filtering	15
Financial dashboard and aggregate queries	20
Monthly budget management and alerts	15
Validation, security, and exception handling	10
UI, testing, and demonstration	10
Total	**100**

Completion goal: Build a working expense management system that can edit records, combine search filters, calculate financial statistics, and warn users when they approach or exceed their budget.